

Instituto Tecnológico de Costa Rica

Ingeniería en Computación

IC-2001 Estructuras de Datos

Mauricio Aviles Cisneros

Informe de Desempeño del Algoritmo Radix Sort

Edwin Andrey Muñoz Sandí

2025075200

22 de mayo del 2026

Condiciones de prueba

Las pruebas se ejecutaron en Visual Studio 2026 sobre Windows, en un equipo con Intel Core i7-9750H y 32 GB de RAM, compilando en modo Release x64. El tiempo se midió con `std::chrono::steady_clock`, abarcando únicamente la llamada a `radixSort`: tanto la generación de la lista como su impresión quedaron fuera de la medición. Los datos de entrada fueron enteros aleatorios en el rango $[0, 10^9]$, generados con `std::mt19937` a partir de una semilla aleatoria (`std::random_device`). Para reducir el ruido del sistema operativo, cada combinación de tamaño y base se ejecutó 5 veces y se reporta el promedio de los tiempos.

Resultados

La Tabla 1 resume los tiempos de ordenamiento en promedio obtenidos al variar el tamaño de la lista y la base utilizada 5 veces.

Table 1: Tiempo de ordenamiento (ms) por tamaño de lista y base.

Tamaño (n)	Base							
	2	10	16	100	256	1000	10000	65536
1 000	4.9772	1.7134	1.4945	0.8634	0.7629	0.7886	2.1065	8.0639
10 000	53.8985	14.3777	12.1118	8.0304	7.4054	5.4381	7.1562	11.3528
100 000	470.0942	151.3638	135.2516	83.5366	76.7366	59.7036	64.2891	55.1351
1 000 000	5853.19	2389.67	2201.896	1613.774	1359.238	1019.0962	1053.252	594.2614

Análisis

Radix Sort ordena haciendo varias pasadas sobre la lista, una por cada dígito del número más grande. La base que se elige influye directamente en cuántas pasadas se necesitan: mientras más alta es la base, menos dígitos tiene cada número y, por lo tanto, menos pasadas hay que dar. Pero esa ventaja viene con un costo. En cada pasada los números se reparten en baldes (uno por cada valor posible de dígito) y al final hay que recorrerlos todos para recoger los elementos en orden. Una base más alta significa más baldes que manejar en cada pasada.

De ahí sale un equilibrio. Una base muy baja, como 2, obliga a dar muchísimas pasadas y resulta lenta. Una base muy alta recorta las pasadas al mínimo, pero la cantidad de baldes empieza a pesar, sobre todo cuando la lista es pequeña y hay pocos elementos para repartir entre tantos baldes. El mejor desempeño aparece donde estos dos efectos se equilibran.

Mis observaciones de los resultados serían las siguientes:

- El crecimiento con n es casi lineal. Para cualquier base, multiplicar n por 10 multiplica el tiempo por aproximadamente 10. Por ejemplo, en base 2:

$$4.98 \rightarrow 53.9 \rightarrow 470 \rightarrow 5853 \text{ ms}$$

Eso confirma que el tiempo es proporcional al tamaño.

- La base 2 es siempre la peor, por mucho. En todos los tamaños es cerca de un orden de magnitud más lenta que la mejor base. En 1 000 000 elementos: 5853 ms vs 594 ms para la base 65536. Esto ocurre por el costo de realizar aproximadamente 30 pasadas.
- La base óptima se desplaza hacia arriba conforme crece la lista:
 - Para $n = 1\,000$, la mejor es base 256 (0.76 ms).
 - Para $n = 10\,000$, la mejor es base 1000 (5.44 ms).
 - Para $n = 100\,000$ y $n = 1\,000\,000$, la mejor es base 65536 (55.1 y 594 ms).
- La base 65536 pasa de ser la peor a ser la mejor. En $n = 1\,000$ es la más lenta de todas (8.06 ms, incluso peor que base 2), porque existen 65536 baldes para repartir apenas 1000 números. Sin embargo, en listas grandes ese costo se vuelve insignificante frente al ahorro de pasadas. El punto de cruce ocurre entre: $n = 10\,000$ y $n = 100\,000$.
- Más base no siempre significa mejor rendimiento. La base 1000 supera a la base 10000 en todos los tamaños. En 1 millón de elementos: 1019 ms vs 1053 ms ambas requieren la misma cantidad de pasadas (3), pero la base 10000 tiene diez veces más baldes que recorrer sin reducir el número de pasadas.

Configuración recomendada

Con los resultados, la configuración más eficiente fue usar base 65536 para conjuntos grandes, pero no existe una única base óptima para todos los casos. En listas pequeñas, usar una base demasiado grande introduce un costo innecesario porque se crean muchísimos baldes que casi no se utilizan. Por eso, mi recomendación sería:

- Para listas pequeñas: usar una base moderada, como 256 o 1000.
- Para listas grandes: usar una base alta, como 65536, porque el ahorro en pasadas compensa el costo adicional.

En general, la mejor configuración es aquella que logra reducir el número de pasadas sin crear una cantidad exagerada de baldes innecesarios.